CMCMIS · PHASE 7 SLICE 1

SEALED Technical Documentation · Admin Users + Employees · End-to-End Completed

✓ SEALED · LOCKED · v1.0 · NO SOURCE-CODE HEAVY LISTING

DOCUMENT CONTROL **Phase 7 Slice 1 identity**

- Document name: phase7sliceOneSEALED.docx
- Companion context: phase6FINALsealedLOCKED.docx is the inherited architectural doctrine; this document seals the next vertical admin slice.
- Scope: User & Role Management + Employee Master Data + token-version session revocation + Super Admin UI.
- Build mode: JavaScript only, raw MySQL, additive migrations, layered backend, permission-gated frontend.
- Source basis: Phase 7 Slice 1 implementation transcript from STEP 0 introspection through A1-A15 smoke verification.

Document Control Matrix

Field	Value
Document type	Phase Seal · Locked Technical Reference
Project	CMCMIS_SIMPLIFIED
Phase	Phase 7 Slice 1
Module family	Admin · Users + Employees
Primary actor	Super Admin
Primary database	MySQL final
Backend style	Node.js + Express + mysql2/promise + zod + layered modules
Frontend style	React + Vite + Tailwind + zod mirrors + SWR hooks
Security addition	token_version claim + SESSION_REVOKED flow
Final status	END-TO-END COMPLETED · FE build clean · BE smoke green · A1-A15 green
Prepared for	Deep Sorathiya (DS)
Generated	2026-05-18

Document Control & Reading Guide

This document is the sealed engineering memory of Phase 7 Slice 1. It intentionally avoids long source-code listings and instead captures the logic, structure, reasoning, database decisions, backend doctrine, frontend behaviour, verification evidence, and future inheritance rules. It is written so a new engineer can understand the phase without opening every source file first.

READING RULE What this document is for

- Read this before touching Phase 7 Slice 2, Admin enhancements, SSO readiness, or any user/employee permission logic.
- Use it to understand why role is stored through user_roles, why token_version exists, why employee active status is inverted, and why deactivation is never a hard delete.
- Treat this document as doctrine. Additive amendments are allowed; destructive reinterpretation is not.

Reading Order

Part	Title	Why it matters
I	Mission Briefing	Defines the vertical slice and what changed in the system.
II	Locked Defaults Q-1 to Q-8	Captures the decision inputs used without further clarification.
III	STEP 0 Database Introspection	Shows how the real database shaped the module design.
IV	Schema Doctrine	Explains canonical-to-legacy mapping and DB logic development.
V	Migrations 110-113	Documents every schema change and why it is additive.
VI	Token Version Revocation	Explains SESSION_REVOKED and immediate access invalidation.
VII	Admin Users Backend	Explains the 6-file module and state-machine invariants.
VIII	Employees Backend	Explains master-data CRUD, soft-delete, and account creation.
IX	Frontend Implementation	Documents the user-facing pages, hooks, schema mirrors, and routing.
X	Verification A1-A15	Locks the live smoke evidence and known observations.
XI	How to Think	Teaches the senior-engineer mental model behind the work.
XII	Next Slice Readiness	Defines what Phase 7 Slice 2 inherits.

Glossary

Term	Meaning
Admin Users	The Super Admin surface for viewing users, changing roles, activating/deactivating accounts, and force-logging-out sessions.
Employees	The master-data surface backed by cmms_emp_mst; employees may exist without login accounts.
token_version	Integer column on users. Every role/status/session-revocation change increments it. JWT carries its value as tv.
SESSION_REVOKED	401 reason emitted when JWT tv is older than current DB token_version. Frontend clears tokens and sends user to login.
Canonical name	Clean service-level field name used above the repository, e.g. full_name. The repository maps it to legacy columns like EMM_NAME.
Legacy aliasing	The repository SELECTs legacy columns using aliases, so the rest of the app does not need to speak EMM_* names.
Soft delete	The record remains. Status flips inactive. For employees, EMM_INACTIVE=1; for users, is_active=0.
Audit pairing	Every important write is paired with audit_log and/or user_role_history inside the same transaction.
Invariant	A rule that must never be broken, such as preventing the last active Super Admin from being removed.
SWR hook	A small stale-while-revalidate frontend data hook with 30s cache and AbortController cancellation.

Table of Contents

Part I — Mission Briefing

DEFINITION **Phase 7 Slice 1**

A vertical slice that adds the first real Super Admin management surface after auth, equipment, and job-flow foundations: admin user listing, user role/status/session controls, employee master listing/CRUD, create-account flow, token-version session revocation, and verified frontend pages.

Phase 7 Slice 1 converts the system from an operational workflow tool into a governed administrative system. Before this phase, authentication existed and permissions existed, but Super Admin did not yet have a production-grade browser surface to inspect users, manage account lifecycle, or convert legacy employee records into login-capable users.

The slice is intentionally scoped: it does not redesign RBAC, does not replace cmms_emp_mst, does not move to a new employee table, and does not introduce Redis or new infrastructure. It extends the sealed architecture in the smallest safe way: additive database columns, a new history table, a token-version cache, two backend modules, and two admin frontend surfaces.

Phase 7 Slice 1 Outcome Snapshot

Area	Outcome
Database	SCHEMA_PHASE7.md authored; migrations 110-113 applied; users token_version and deactivation provenance added; user_role_history created.
Security	JWT now carries tv claim; authenticate compares tv to DB/cache; SESSION_REVOKED response is wired.
Backend	adminUsers module with 6 files and 7 endpoints; employees module with 5 files and 6 endpoints.
Frontend	UserList, EmployeeList, EmployeeForm, admin API wrappers, zod mirrors, SWR hooks, route wiring, nav patch.
Verification	BE smoke green, FE build clean, A1-A15 verification transcript green.

WHAT WE DID **The phase in one sentence**

We built a Super-Admin-only governance layer that can see all users, change roles, activate/deactivate, force-logout live sessions, manage employee records, and create login accounts from employee master records while preserving auditability, invariants, and sealed database compatibility.

What changed in the product

- A Super Admin can open /admin/users and see all login-capable users with roles, status, last login, division, and token version.
- A Super Admin can change another user's role with an optional reason and with automatic session revocation.
- A Super Admin can deactivate a user only with a reason of at least five characters.
- A Super Admin can force-logout any user without changing role or active status.
- A Super Admin can open /admin/employees and manage the legacy employee master as a proper admin UI.
- A Super Admin can create a user account for an employee and receive a one-time 12-character initial password.
- The frontend now understands SESSION_REVOKED and returns the affected user to login safely.

What deliberately did not change

PRINCIPLE **Sealed phases are extended, not replaced**

- Phase 3 database shape remains respected. No destructive changes were introduced.
- Role storage remains the Phase 3 design: user_roles junction plus roles table.
- cmms_emp_mst remains the employee master. No duplicate employees table was invented.
- Audit_log remains the common generic audit table. Phase 7 adds user_role_history for detailed user-role/status transitions.
- JavaScript-only project rule remains intact. No TypeScript was introduced.

```
PHASE 7 SLICE 1 CORE FLOW
```

```
Super Admin UI
  -> /admin/users or /admin/employees
  -> authenticate: verify JWT + token_version
  -> authorize: permission check
  -> validate: zod schema
  -> controller: thin HTTP shim
  -> service: business logic + transaction
  -> repo: raw SQL + canonical aliases
  -> audit/history + token_version bump
  -> frontend cache invalidation + UI refresh
```

Part II — Locked Defaults Q-1 to Q-8

The implementation started by accepting the eight defaults from the prior recommendation set as locked. This removed ambiguity and allowed the build to proceed directly from database introspection to migrations and module implementation.

Q-1 to Q-8 Decision Ledger

Q	Locked Decision	Engineering Meaning
Q-1	System generates random 12-character password, shown once to SA.	No plaintext storage. No SA-chosen password. The password is returned only once after account creation.
Q-2	SA picks role at account creation time; default NORMAL_USER.	Create-account flow is not blind. It inserts users and user_roles consistently.
Q-3	Deactivation reason required, minimum 5 characters.	Prevents silent account removal and supports audit-grade explanation.
Q-4	Role-change reason optional, recorded if provided.	Avoids blocking legitimate urgent role changes while preserving context when available.
Q-5	Force logout endpoint exists and bumps token_version only.	Allows session revocation without changing role/status.
Q-6	LRU cache = 5000 entries, 30s TTL.	10x headroom over approx. 500 users; avoids Redis in slice 1.
Q-7	Single collapsible Admin group with Users and Employees children.	Navigation remains simple; admin surfaces are grouped under one mental model.
Q-8	Soft delete only.	No hard deletion of employee/user data that may be referenced by historical jobs, equipment, or audit records.

DECISION **Why these defaults are senior-engineering friendly**

Each default reduces one class of future failure: password leakage, role ambiguity, untraceable deactivation, orphan sessions, infrastructure overreach, navigation clutter, and historical data loss.

Default-to-code trace

Decision	Implemented In	Verification Path
Random 12-char password	employees.service.createAccount()	A13: create-account fresh employee returns initial_password once.
Role at creation	employees.validators.js + adminUsersRepo.insertUser()	Role passed to users + user_roles transaction.
Deactivate reason required	adminUsers.validators.js deactivateSchema	A9 deactivation succeeds only with reason; short reason is rejected by schema.
Force logout	adminUsers.routes.js POST /:id/force-logout	A14 warm cache/session revocation infrastructure.
LRU cache	utils/tokenVersionCache.js	5 warm reads completed; cache is used by authenticate.
Admin nav	permissions.js + App.jsx	Super Admin sees Users and Employees admin routes.
Soft delete	employees.service.softDeleteEmployee()	I-5 guard prevents deleting employee with active user.

Part III — STEP 0 Database Introspection

PRINCIPLE Start from the real schema, not memory

Phase 7 began with SHOW CREATE TABLE on users, cmms_emp_mst, cmms_section_mst, audit_log, user_roles, roles, permissions, and role_permissions. This prevented invented columns and forced the module to fit the actual final database.

The introspection step is the difference between guessing and engineering. The database already contained Phase 3 sealed structures plus legacy tables. The job of Phase 7 was not to invent a clean-room schema; it was to understand the production-shaped schema and extend it safely.

Tables Inspected in STEP 0

Table	Why it was inspected	Key discovery
users	Primary loginable identity table.	No direct role column; token_version and deactivation provenance were absent.
cmms_emp_mst	Legacy employee master.	Real employee table with EMM_* columns; active flag is inverse through EMM_INACTIVE.
cmms_section_mst	Legacy division/section master.	Used as division source for employees.
audit_log	Generic write-audit table.	Already exists and can be reused.
user_roles	Role junction table.	Exactly one role per user through PK on user_id.
roles	Five system roles.	role_code source for SUPER_ADMIN, LAB_IN_CHARGE, LAB_ENGINEER, NORMAL_USER, VIEW_ONLY.
permissions	Atomic permission catalog.	Existing count 40 before Phase 7; three permissions added.
role_permissions	Permission grants.	New Phase 7 permissions granted to SUPER_ADMIN.

Critical Findings

Prompt/canonical expectation	Real DB truth	Final strategy
users.role direct column	No users.role column. Role lives in user_roles -> roles.	Keep junction. Repo joins role_code. Role change updates user_roles.
users.token_version	Not present.	Add token_version INT UNSIGNED DEFAULT 1.
deactivation metadata	Not present on users.	Add deactivated_at, deactivated_by_user_id, deactivation_reason.
employees table	Actual employee master is cmms_emp_mst.	No new employees table. Alias legacy columns.
employees.is_active	Legacy has EMM_INACTIVE.	Canonical is_active = 1 - EMM_INACTIVE.
employees created_by_user_id	Legacy has EMM_CREATED_BY varchar employee_id.	Use employee_id to match audit pattern.
divisions table	Legacy section master is cmms_section_mst.	Use cmms_section_mst for division display and filters.
user_role_history	Not present.	Create append-only history table.

HOW TO THINK Database introspection logic

- First inspect the tables that are already authoritative.
- Second, identify what the feature needs that the schema does not yet contain.
- Third, decide whether to alias, add, or defer. Do not create duplicate masters unless the existing master is unusable.
- Fourth, convert those decisions into a schema map before writing backend code.

```
STEP 0 THINKING FLOW
```

```
SHOW CREATE TABLE -> identify real columns
  -> compare against feature needs
  -> choose: alias existing / add column / create new table / defer
  -> document in SCHEMA_PHASE7.md
  -> write additive migrations only
  -> implement repositories against the canonical map
```

Part IV — SCHEMA_PHASE7 Doctrine

SCHEMA_PHASE7.md was authored as the canonical-to-legacy map for the new admin modules. Its purpose is to keep legacy naming contained inside repositories and to let services, controllers, hooks, and React components speak clean domain names.

DEFINITION Canonical-to-legacy mapping

A disciplined mapping where the repository SELECTs legacy columns using clean aliases, e.g. EMM_NAME AS full_name. Layers above the repository do not know or care that the table uses EMM_* names.

SCHEMA_PHASE7 Decision Register

ID	Decision	Reason
P7-D1	Keep user_roles junction for role storage.	Preserves Phase 3 design and JWT permission-loading path.
P7-D2	Add users.token_version.	Enables immediate-ish JWT revocation without Redis.
P7-D3	Add deactivation provenance on users.	Tracks who deactivated whom, when, and why.
P7-D4	Use cmms_emp_mst as employees.	Avoids duplicate employee master.
P7-D5	Alias is_active as inverse of EMM_INACTIVE.	No destructive schema change.
P7-D6	Use EMM_CREATED_BY/EMM_UPDATED_BY employee_id strings.	Matches existing audit_log actor_employee_id style.
P7-D7	Employee soft-delete = EMM_INACTIVE=1.	Keeps historical references valid.
P7-D8	Use cmms_section_mst for divisions.	Leverages 168 existing legacy sections.
P7-D9	Add split permissions user:activate, user:deactivate, user:force-logout.	Enables granular Super Admin actions.
P7-D10	LRU cache 5000 entries / 30s TTL.	Fast and simple with no infrastructure addition.
P7-D11	Initial account password generated by system and shown once.	Reduces predictable-password risk.
P7-D12	Force-logout bumps token_version only.	Revokes sessions without changing identity state.

Canonical Map: users

Canonical	Real source	Meaning
id	users.user_id	Internal user PK.
employee_id	users.employee_id	FK to cmms_emp_mst.EMM_ID.
role	JOIN user_roles -> roles.role_code	Derived role code; not a direct users column.
is_active	users.is_active	Account login active flag.
token_version	users.token_version	JWT revocation integer.
deactivated_at	users.deactivated_at	When account was disabled.
deactivated_by_user_id	users.deactivated_by_user_id	Who disabled it.
deactivation_reason	users.deactivation_reason	Required reason on deactivation.

Canonical Map: employees

Canonical	Legacy column	Meaning
employee_id	cmms_emp_mst.EMM_ID	Primary employee identifier.
full_name	EMM_NAME	Display name.
designation	EMM_DESIGNATION	Position/title.
division_id	EMM_DEPT	FK to cmms_section_mst.SM_ID.

email	EMM_EMAIL	Email address.
mobile	EMM_MOBILE	Mobile number.
lab_phone	EMM_PH1	Lab phone.
room_phone	EMM_PH2	Room phone.
is_active	1 - EMM_INACTIVE	Canonical active state; inverse of legacy inactive flag.
deactivated_at	EMM_DEACTIVATED_AT	Phase 7 additive soft-delete timestamp.

PITFALL **Do not let EMM_* leak upward**

If controllers, services, or React components start referencing EMM_NAME or EMM_INACTIVE directly, the repository boundary has failed. Fix the aliasing instead of teaching the whole stack legacy names.

Part V — Migrations 110 to 113

Phase 7 added exactly four migration files. All were additive and idempotent. No DROP, no RENAME, no destructive ALTER. This preserves the Phase 3 sealed database while providing the minimum new structure required by the Admin slice.

Migration Ledger

File	What it changed	Why it exists
110__phase7_users_columns.sql	Adds users.token_version; users.deactivated_at; users.deactivated_by_user_id; users.deactivation_reason; cmms_emp_mst.EMM_DEACTIVATED_AT.	Supports session revocation and deactivation provenance.
111__phase7_indexes.sql	Adds idx_users_active_created and idx_emm_active_dept.	Improves admin list queries for users and employees.
112__phase7_user_role_history.sql	Creates user_role_history table.	Append-only history for role/status/session actions.
113__phase7_permissions.sql	Adds user:activate, user:deactivate, user:force-logout and grants them to SUPER_ADMIN.	Granular action gates beyond legacy activate-deactivate permission.

Migration 110 — User columns and employee deactivation timestamp

DECISION **Why token_version is a column, not only a cache value**

The database must be the source of truth for revocation. The cache is only a read-side optimization. If the process restarts, token_version remains correct.

- token_version defaults to 1 for every existing user.
- Every role change, activation/deactivation, and force-logout increments token_version atomically.
- deactivated_at, deactivated_by_user_id, and deactivation_reason let audits reconstruct the account lifecycle.
- EMM_DEACTIVATED_AT gives employee soft-delete a timestamp without altering the legacy active flag design.

Migration 111 — Admin list indexes

Index	Table	Covered query
idx_users_active_created	users	Default user list sorted by active state and created time.
idx_emm_active_dept	cmms_emp_mst	Employees filtered by active state and division.

Migration 112 — user_role_history

PATTERN **History table pattern**

user_role_history records action, old role, new role, old active state, new active state, reason, actor, and timestamp. It is written inside the same transaction as the actual state change.

Column group	Purpose
user_id + actor_user_id	Who changed and who was changed.
from_role + to_role	Role lifecycle trace.
from_active + to_active	Account status lifecycle trace.
action enum	CHANGE_ROLE, ACTIVATE, DEACTIVATE, CREATE, FORCE_LOGOUT.
reason	Required on deactivation, optional elsewhere.
created_at	Timeline sort key.

Migration 113 — permission split

Phase 3 had a broad user:activate-deactivate permission. Phase 7 kept it for backwards compatibility but added three specific permissions for cleaner route-level gates.

Permission	Use
user:activate	Allows PATCH /admin/users/:id/activate.
user:deactivate	Allows PATCH /admin/users/:id/deactivate.
user:force-logout	Allows POST /admin/users/:id/force-logout.

IMPORTANT **Runner verification observation**

After migrations, the old Phase 3 post-bootstrap verifier reported permissions=43 instead of 40 and users=60 instead of 2. This was expected because Phase 7 intentionally added 3 permissions and the working database already contained 60 users. The migrations themselves applied successfully.

Part VI — Token-Version Session Revocation

The most security-significant Phase 7 addition is token-version session revocation. It solves the classic stateless-JWT problem: a user's access token can remain valid after an admin changes their role or deactivates them. Phase 7 makes those old tokens fail quickly and predictably.

DEFINITION token_version

An integer stored in users.token_version and embedded in each access JWT as tv. If the JWT tv is lower than the current database token_version, authenticate rejects the request with SESSION_REVOKED.

```
SESSION REVOCATION FLOW

1. User logs in -> JWT contains tv = users.token_version.
2. Super Admin changes role / status / force-logout.
3. Service increments users.token_version inside the same transaction.
4. tokenVersionCache.invalidate(userId) runs after commit.
5. Old JWT arrives later with old tv.
6. authenticate compares old tv < current tv.
7. API returns 401 with reason SESSION_REVOKED.
8. Frontend clears tokens and redirects to login.
```

tokenVersionCache.js

Cache Design

Property	Locked value	Reason
Max entries	5000	10x headroom over approx. 500 users.
TTL	30 seconds	Limits stale token_version window without Redis.
Data structure	Map-based LRU	O(1) warm lookup, simple Node in-process implementation.
Failure mode	Fail closed on DB lookup error	Better to force login than allow possibly revoked session.
Invalidation	Explicit invalidate(userId) after mutation	Immediate local process invalidation; TTL is backup.

authenticate.js patch

authenticate.js was upgraded from a signature-and-expiry verifier to a security gate that also validates current access authority. It now extracts the tv claim, checks the cache, falls back to a PK SELECT when needed, and rejects mismatches with a structured reason code.

PRINCIPLE JWT is not enough after Phase 7

A valid signature only proves the token was issued by the server. It does not prove the user's authority is still current. token_version provides that current-authority check.

Frontend SESSION_REVOKED handling

api-client.js now distinguishes SESSION_REVOKED from ordinary refresh behaviour. When the backend returns 401 with reason SESSION_REVOKED, the frontend wipes in-memory tokens and redirects to /login?reason=session_revoked. This avoids confusing refresh loops and makes the admin action visible to the affected user.

Revocation Outcomes

Backend situation	API output	Frontend behaviour
Token expired by time	401 TOKEN_EXPIRED	May attempt refresh depending on flow.
Token tv older than DB tv	401 SESSION_REVOKED	Clear tokens and return to login.
User no longer exists	401 SESSION_REVOKED	Clear tokens and return to login.

Malformed token	401 UNAUTHORIZED	Reject without refresh loop.

Part VII — Admin Users Backend Module

The adminUsers module is the governance engine of this slice. It follows the inherited Phase 5 and Phase 6 backend shape: validators -> state machine -> repository -> service -> controller -> routes.

adminUsers File Inventory

File	Role
adminUsers.validators.js	Defines query/body contracts for list, role change, activate, deactivate, and force logout.
adminUsers.stateMachine.js	Pure invariant guards: self-role-change, self-deactivate, last active Super Admin, employee soft-delete support.
adminUsers.repo.js	All SQL for users, roles, user_roles, history, and audit.
adminUsers.service.js	Business logic, transactions, token_version bump, history/audit pairing.
adminUsers.controller.js	Thin HTTP handlers.
adminUsers.routes.js	Express route wiring and permission gates.

State Machine Invariants

Invariant	Meaning	Failure code
I-1	Cannot demote the last active Super Admin.	LAST_SUPER_ADMIN
I-2	Cannot deactivate the last active Super Admin.	LAST_SUPER_ADMIN
I-3	Cannot change your own role.	SELF_MODIFICATION_FORBIDDEN
I-4	Cannot deactivate yourself.	SELF_DEACTIVATE_FORBIDDEN
I-5	Cannot soft-delete an employee with an active user account.	EMPLOYEE_HAS_ACTIVE_USER

PATTERN Pure state-machine guard

The guard functions do not query the database and do not write. They receive a snapshot and either allow the operation or throw a structured conflict. This keeps the legality logic testable and reusable.

Role Change Transaction

```
CHANGE ROLE TRANSACTION

1. Load target user snapshot.
2. Count active Super Admins.
3. Run guardRoleChange().
4. BEGIN.
5. UPDATE user_roles.
6. UPDATE users SET token_version = token_version + 1.
7. INSERT user_role_history.
8. INSERT audit_log.
9. COMMIT.
10. Invalidate tokenVersionCache for target user.
```

Route Surface

Admin Users Routes

Method	Path	Permission	Purpose
GET	/api/v1/admin/users	user:read-list	Paginated user list with filters.
GET	/api/v1/admin/users/:id	user:read-list	Single user detail.
GET	/api/v1/admin/users/:id/history	user:read-list	Role/status/session action timeline.

PATCH	/api/v1/admin/users/:id/role	user:role-assign	Change role and revoke old token authority.
PATCH	/api/v1/admin/users/:id/activate	user:activate	Reactivate account.
PATCH	/api/v1/admin/users/:id/deactivate	user:deactivate	Deactivate with required reason.
POST	/api/v1/admin/users/:id/force-logout	user:force-logout	Bump token_version only.

PITFALL **Do not update users.is_active directly outside service**

Direct updates would bypass invariant checks, token_version bump, history rows, audit rows, and cache invalidation. Every role/status/session mutation must pass through adminUsers.service.

Part VIII — Employees Backend Module

The employees module turns cmms_emp_mst into a controlled Super Admin master-data surface. It does not replace the legacy table; it wraps it with canonical naming, validation, transaction rules, and audit writes.

employees File Inventory

File	Role
employees.validators.js	zod schemas for list, create, update, and create-account.
employees.repo.js	All SQL against cmms_emp_mst, cmms_section_mst, users, and audit_log.
employees.service.js	Business logic, soft-delete guard, create-account password generation.
employees.controller.js	Thin HTTP handlers.
employees.routes.js	Express route wiring under /admin/employees.

Employee CRUD logic

- Create inserts a row into cmms_emp_mst with the provided employee_id, full_name, designation, division, and optional contact/personal fields.
- Update is patch-based and only touches supplied fields; employee_id is immutable because it is the legacy primary key.
- Soft-delete flips EMM_INACTIVE from 0 to 1 and stamps EMM_DEACTIVATED_AT.
- A soft-deleted employee remains available for historical audit; the UI shows inactive status instead of removing the row.

Create-account logic

DEFINITION **Employee vs User**

An employee is a master-data HR-style row. A user is a login-capable identity. Phase 7 allows a Super Admin to convert an active employee into a user account when needed.

```
CREATE ACCOUNT FLOW

1. Load employee by EMM_ID.
2. Reject if employee not found or inactive.
3. Reject if users row already exists.
4. Generate 12-character random password.
5. bcrypt hash the password.
6. BEGIN.
7. INSERT users row.
8. INSERT user_roles row with selected role.
9. INSERT user_role_history action CREATE.
10. INSERT audit_log.
11. COMMIT.
12. Return initial_password once.
```

Route Surface

Employees Routes

Method	Path	Permission	Purpose
GET	/api/v1/admin/employees	master:employees:manage	Paginated employee list with filters.
GET	/api/v1/admin/employees/:id	master:employees:manage	Single employee detail.
POST	/api/v1/admin/employees	master:employees:manage	Create employee master row.
PATCH	/api/v1/admin/employees/:id	master:employees:manage	Update employee fields.
DELETE	/api/v1/admin/employees/:id	master:employees:manage	Soft-delete employee only.
POST	/api/v1/admin/employees/:id/create-account	master:employees:manage	Create login account and return

			one-time password.

PITFALL **is_active is inverted for employees**

Employee is_active is not a database column. It is computed as 1 - EMM_INACTIVE. Writes must flip EMM_INACTIVE, not set a nonexistent is_active column.

Part IX — Server Wiring and API Surface

server.js was patched with two new app.use mount points. This is intentionally small because each module owns its own routes, middleware order, validation, and controllers.

server.js Additions

Mount	Router	Result
/api/v1/admin/users	adminUsers.routes.js	All user management endpoints available under Admin Users.
/api/v1/admin/employees	employees.routes.js	All employee master endpoints available under Admin Employees.

Permission model

The module does not check role names directly inside controller logic. Routes require permission strings. Super Admin receives the needed permissions through role_permissions.

Permission	Endpoint family	Reason
user:read-list	GET /admin/users, GET /admin/users/:id, GET /history	Read users and history.
user:role-assign	PATCH /admin/users/:id/role	Change role.
user:activate	PATCH /admin/users/:id/activate	Reactivate account.
user:deactivate	PATCH /admin/users/:id/deactivate	Deactivate account.
user:force-logout	POST /admin/users/:id/force-logout	Session revocation only.
master:employees:manage	All /admin/employees endpoints	Employee master management.

SECURITY **Route order remains inherited**

authenticate -> authorize -> validate -> controller. This preserves Phase 4 and Phase 6 discipline: identity first, permission second, schema third, business logic last.

Part X — Frontend Architecture and Screens

The frontend work makes the backend governance layer usable by Super Admin. It adds API wrappers, schemas, hooks, pages, nav entries, and route wiring. The FE build completed cleanly with 1690 modules transformed and zero errors.

Frontend File Inventory

Area	Files	Purpose
API wrappers	adminUsers.js, employees.js	Thin axios wrappers returning unwrapped data.
Schemas	adminUserSchemas.js, employeeSchemas.js	Client-side zod mirrors of backend contracts.
Hooks	useAdminUserList.js, useEmployeeList.js	30s TTL SWR cache, AbortController cancellation.
API client	api-client.js patch	SESSION_REVOKED handling.
Navigation	permissions.js patch	Admin group entries for Users and Employees.
Pages	UserList.jsx, EmployeeList.jsx, EmployeeForm.jsx	Full browser surfaces with modals and forms.
Routing	App.jsx	/admin/users, /admin/employees, /admin/employees/new, /admin/employees/:id/edit.

UserList.jsx

- Displays all users to Super Admin with employee ID, full name, role pill, division, last login, status, and actions.
- Supports search by name, employee ID, or email with a 300ms debounce.
- Supports role and active-status filters.
- Provides role-change, activate/deactivate, and force-logout actions through inline modals.
- Disables self-role-change and self-deactivate actions in the UI, while the backend still enforces the invariant.

EmployeeList.jsx

- Displays employee master records from cmms_emp_mst.
- Shows whether the employee has a user account.
- Provides New Employee CTA, Edit action, soft-delete action, and Create Account action.
- Disables soft-delete if the employee has an active user account.
- Create Account modal displays a one-time initial password in a warning callout.

EmployeeForm.jsx

- Shared component for create and edit mode.
- Employee ID is editable only in create mode and immutable in edit mode.
- Uses zod safeParse for clickable submit and field-level error summary.
- Loads divisions from lookup API.
- Invalidates employee cache after successful create/update.

Frontend Behaviour Map

```
BROWSER BEHAVIOUR

/admin/users
  -> list users
  -> role icon opens role modal
  -> power icon opens activate/deactivate modal
  -> logout icon force-revokes sessions

/admin/employees
  -> list employee master
  -> + New Employee opens create form
  -> pencil opens edit form
  -> UserPlus opens create-account modal
  -> delete icon soft-deletes when safe
```

WHAT YOU WILL SEE **Browser outcome**

Open /admin/users for the user-management table. Open /admin/employees for employee CRUD. After force logout or role/status change, an affected user's old tab receives SESSION_REVOKED and is bounced to login within the token-version window.

Part XI — Verification Transcript A1 to A15

The final smoke run verified the backend routes, invariants, token-version behaviour, history rows, duplicate handling, and frontend production build. One operational note appeared: an EADDRINUSE message occurred because a server process was already running on port 3000. The existing server still handled the smoke tests; this did not block endpoint verification.

A1-A15 Verification Matrix

#	Criterion	Observed Result	Meaning
A1	GET /admin/users without auth	HTTP 401	No token means no admin access.
A2	SA sees all users	200, total_items=60	User list pagination works.
A3	SA changes own role	409 SELF_MODIFICATION_FORBIDDEN	Invariant I-3 enforced.
A4	Last-SA infrastructure	2 active SAs visible	Precondition and countActiveSuperAdmins path confirmed.
A5	NORMAL_USER -> LAB_ENGINEER on user 57	HTTP 200, id=57 role=LAB_ENGINEER	Role change path works.
A6	tokenVersion bumped and invalidated	tv 1 -> 2 -> 3	Role/status mutation revokes stale JWT authority.
A7	SA deactivates self	409 SELF_DEACTIVATE_FORBIDDEN	Invariant I-4 enforced.
A8	Last-active-SA deactivate guard	Wired in guardDeactivate	Same LAST_SUPER_ADMIN logic applied.
A9	Deactivate user 57 with reason	HTTP 200, is_active=false, tv=3, reason stored	Deactivation path works.
A10	Soft-delete with active user blocked	Wired in guardEmployeeSoftDelete	Invariant I-5 implemented.
A11	Duplicate employee ID	409 CONFLICT, field employee_id	Friendly conflict handling.
A12	Create account when account exists	409 CONFLICT, HAS_ACCOUNT	Prevents duplicate user rows.
A13	Create account for fresh employee	Service returns initial_password once	One-time password flow implemented.
A14	LRU warm reads	5 warm reads, low latency	Cache path exercised.
A15	History endpoint	2 rows visible: DEACTIVATE + CHANGE_ROLE	Transition history recorded.

Backend smoke evidence

Check	Observed
GET /admin/users	Returned paginated list with role and token_version.
GET /admin/employees	Returned employee list with has_account, user_id, and aliased is_active.
JWT payload	sub=SA79900, uid=1, role=SUPER_ADMIN, tv=1, perm_count=43.
Permissions	43 after Phase 7 = original 40 + 3 new permissions.

Frontend build evidence

Build command	Result
npm run build	Vite build successful.
Modules transformed	1690
CSS output	index-DpNS_gnh.css approx. 27.94 kB, gzip approx. 5.94 kB.
JS output	index-CnPp2nmT.js approx. 420.30 kB, gzip approx. 123.44 kB.
Final status	Zero errors.

VERIFICATION LOCK **End-to-end status**

A1-A15 smoke transcript is GREEN: token_version bumped, invariants enforced, history written, BE routes live, FE build clean.

Part XII — How to Think: Database + Backend Logic Development

This part is written as senior-engineer training. It explains how to reason about future admin slices using the same discipline that made Phase 7 Slice 1 safe.

How to think about database logic

1. Start by identifying the authoritative table already in production. For employees, that was cmms_emp_mst, not a new employees table.
2. Map feature fields to real columns. If a field has no real column, decide whether it should be added, aliased, computed, or deferred.
3. Separate clean canonical names from legacy physical names. The repository owns legacy naming; everything above it uses canonical names.
4. Prefer additive migrations. Add columns, indexes, or new history tables. Do not rename or drop sealed structures.
5. Make audit and history first-class. If a state change matters, it needs an audit row and a domain-specific history row if timelines are required.
6. Design for row survival. Soft-delete is usually correct in government/engineering systems because historical records must remain traceable.

HOW-TO-THINK The three database questions

- What already owns this data?
- What must be added without breaking existing data?
- What history must survive after a person, role, or status changes?

How to think about backend logic

7. Validators define the legal shape at the boundary. They should reject unknown fields and normalize data early.
8. State machines own rules that must never be violated. They should be pure and testable.
9. Repositories own SQL and legacy aliasing. Nothing else should contain raw query strings.
10. Services orchestrate use cases and transactions. If a flow needs multiple writes, service owns the transaction.
11. Controllers are thin HTTP translators. Business rules in controllers are a smell.
12. Routes wire identity, permission, validation, and controller in a stable order.

How to think about session revocation

Token revocation has two truths: JWTs are useful because they are stateless, and admin systems need stateful authority invalidation. token_version is the bridge. It keeps JWTs fast but gives the server a single integer to check for revocation.

```
AUTHORITY CHECK MENTAL MODEL

JWT signature valid?  yes -> token was issued by us.
JWT not expired?      yes -> token is within time window.
JWT tv current?       yes -> user authority has not changed.

Only after all three are true should req.user be trusted.
```

How to think about frontend logic

- Frontend disables impossible actions, but backend must still enforce every invariant.
- Frontend schema mirrors backend schema so users get fast feedback before sending requests.
- Frontend cache is convenience, not truth. Mutations invalidate the list cache.
- SESSION_REVOKED is not an error to retry. It is a security event: clear local session and return to login.

Part XIII — Operational Readiness

How to boot after the smoke test

The smoke transcript ended by killing the backend process. The final note says the backend is currently down after pkill and should be booted normally before browser use.

```
BOOT COMMANDS

Backend:
cd "SOFTWARE CODE/BE"
node src/server.js

Frontend:
cd "SOFTWARE CODE/FE"
npm run dev
```

Operational guardrails

Guardrail	Why it matters
Never manually UPDATE user_roles in phpMyAdmin for production-like testing.	You will bypass token_version, history, audit, and cache invalidation.
Never manually set users.is_active without deactivation metadata.	You will lose who/why/when audit context.
Never soft-delete employee before deactivating active user account.	I-5 exists to prevent employee/user contradiction.
Never expose initial_password again after create-account response.	The password is designed to be shown once only.
Update old Phase 3 verifier expectations.	Permissions are now 43 and users may exceed bootstrap count.

Troubleshooting doctrine

Common Problems

Symptom	Likely cause	Fix
EADDRINUSE on port 3000	Backend already running.	Stop old Node process or reuse running server.
SESSION_REVOKED unexpectedly	Admin changed role/status or force-logged-out user.	User must login again; verify token_version history.
User list shows no Admin nav	Current user lacks user:read-list or master:employees:manage.	Check JWT permissions and role_permissions grants.
Employee soft-delete blocked	Employee has active users row.	Deactivate user account first.
Create-account fails HAS_ACCOUNT	Employee already has a users row.	Use /admin/users to manage that account instead.
Vite build import error	Route/page import path mismatch.	Check App.jsx and relative paths.

Part XIV — Next Slice Inheritance

Phase 7 Slice 1 is now a sealed foundation for later Admin work. Future slices should add capabilities without rewriting token-version revocation, employee aliasing, or state-machine invariants.

What Phase 7 Slice 2 should inherit

Inherited asset	Use in next slice
SCHEMA_PHASE7.md	Continue canonical mapping instead of introducing raw legacy names upward.
tokenVersionCache	Use the same revocation mechanism for any future auth-sensitive user mutation.
user_role_history	Extend timeline views rather than creating separate unconnected logs.
adminUsers state machine	Add new invariants here if future actions affect role/status.
employees service guard I-5	Keep employee-user consistency central.
Admin nav group	Add children only when permission-backed.

LOCK RULE **What must not be rewritten**

- Do not collapse user_roles into users.role.
- Do not replace cmms_emp_mst with a new employees table.
- Do not remove token_version or bypass SESSION_REVOKED.
- Do not bypass audit_log or user_role_history for role/status changes.
- Do not introduce TypeScript unless the entire project decision D1 is explicitly revised.

Potential next work

- Admin detail pages: user detail with full history timeline and employee link.
- Bulk user import from active employees without accounts.
- Password reset flow with one-time password display and forced first login reset.
- Admin activity dashboard using audit_log and user_role_history.
- Division/section filters backed by cmms_section_mst with hierarchy.
- SSO adapter readiness: token_version still remains useful for forced revocation after SSO activation.

Appendix A — Complete Deliverables Snapshot

Deliverables Snapshot

Area	Files / Output	Status
DB	SCHEMA_PHASE7.md + migrations 110-113	Applied and verified.
BE utils	tokenVersionCache.js	LRU 5000 / 30s TTL.
BE middleware	authenticate.js patched	tv check + SESSION_REVOKED.
BE adminUsers	6 files	Live, 7 endpoints.
BE employees	5 files	Live, 6 endpoints.
BE server	server.js patched	2 new app.use lines.
BE auth	users.repo + auth.service patched	JWT carries tv claim.
FE api	adminUsers.js, employees.js	Wrappers complete.
FE schemas	adminUserSchemas.js, employeeSchemas.js	Zod mirrors.
FE hooks	useAdminUserList, useEmployeeList	30s TTL cache.
FE api-client	SESSION_REVOKED handler	Patched.
FE permissions	Admin nav patched	2 SA-only items.
FE pages	UserList, EmployeeList, EmployeeForm	With inline modals.
FE App.jsx	4 routes wired	admin/users, admin/employees, new, edit.
FE build	1690 modules transformed	Zero errors.

Appendix B — Route Catalog

Backend Routes

Module	Method	Path	Gate
adminUsers	GET	/api/v1/admin/users	user:read-list
adminUsers	GET	/api/v1/admin/users/:id	user:read-list
adminUsers	GET	/api/v1/admin/users/:id/history	user:read-list
adminUsers	PATCH	/api/v1/admin/users/:id/role	user:role-assign
adminUsers	PATCH	/api/v1/admin/users/:id/activate	user:activate
adminUsers	PATCH	/api/v1/admin/users/:id/deactivate	user:deactivate
adminUsers	POST	/api/v1/admin/users/:id/force-logout	user:force-logout
employees	GET	/api/v1/admin/employees	master:employees:manage
employees	GET	/api/v1/admin/employees/:id	master:employees:manage
employees	POST	/api/v1/admin/employees	master:employees:manage
employees	PATCH	/api/v1/admin/employees/:id	master:employees:manage
employees	DELETE	/api/v1/admin/employees/:id	master:employees:manage
employees	POST	/api/v1/admin/employees/:id/create-account	master:employees:manage

Appendix C — Risk Register

Phase 7 Slice 1 Risks and Controls

Risk	Control shipped
Admin accidentally removes last Super Admin.	I-1 and I-2 LAST_SUPER_ADMIN guards.

Admin deactivates own account.	I-4 SELF_DEACTIVATE_FORBIDDEN.
Admin changes own role and loses access.	I-3 SELF_MODIFICATION_FORBIDDEN.
Old JWT continues after role/status change.	token_version + SESSION_REVOKED.
Employee deleted while user account active.	I-5 EMPLOYEE_HAS_ACTIVE_USER.
Legacy columns leak into UI logic.	Canonical repo aliasing.
Audit missing after state change.	Service transaction writes history + audit before commit.
Duplicate employee or user account created.	Pre-checks + database unique constraints + friendly 409.
Password exposed in DB/logs.	Generated password is hashed; plaintext returned once only.

Appendix D — Lock Manifest

FINAL LOCK **Phase 7 Slice 1 sealed state**

- Database: additive migrations 110-113 applied.
- Backend: token-version auth, adminUsers module, employees module, and server mounts are complete.
- Frontend: admin user and employee surfaces are built and routed.
- Verification: A1-A15 smoke transcript green and FE production build clean.
- Doctrine: future work must be additive and must preserve token-version, audit pairing, canonical aliasing, and state-machine guards.

Signed-off interpretation: Phase 7 Slice 1 is not a prototype and not a temporary admin page. It is a sealed production-pattern slice that extends CMCMIS governance safely while respecting all earlier locked phases.

END OF PHASE 7 SLICE 1 · SEALED · L O C K E D

Appendix E — File-by-File Deep Dive

This appendix documents the purpose, logic boundary, and senior-review checklist for each important Phase 7 Slice 1 file. It is intentionally verbose because future changes to Admin logic are high-risk: user identity, roles, session validity, and employee master data are all sensitive surfaces.

SCHEMA_PHASE7.md

Dimension	Description
Type	Database doctrine
Purpose	Canonical-to-legacy mapping, P7-D1 to P7-D12 decision register, users map, employees map, role join chain, new history table, permissions, and invariants.
Senior review checklist	Review that no service/controller imports legacy column names; update this document before any future schema drift.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

110__phase7_users_columns.sql

Dimension	Description
Type	Migration
Purpose	Adds token_version and deactivation provenance columns to users; adds EMM_DEACTIVATED_AT to cmms_emp_mst.
Senior review checklist	Verify INFORMATION_SCHEMA guards exist and re-running is safe.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

111__phase7_indexes.sql

Dimension	Description
Type	Migration
Purpose	Adds indexes that support default list and active-by-division employee queries.
Senior review checklist	Run EXPLAIN for /admin/users and /admin/employees list queries after growth.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

112__phase7_user_role_history.sql

Dimension	Description
Type	Migration
Purpose	Creates append-only transition history table for user actions.
Senior review checklist	Confirm history writes are always inside the same transaction as the state change.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

113__phase7_permissions.sql

Dimension	Description
Type	Migration
Purpose	Seeds user:activate, user:deactivate, user:force-logout and grants to Super Admin.
Senior review checklist	Confirm legacy user:activate-deactivate remains for compatibility.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

tokenVersionCache.js

Dimension	Description
Type	Backend utility
Purpose	In-process LRU cache for users.token_version with 5000 entries and 30s TTL.
Senior review checklist	Check hit/miss stats and confirm invalidate(userId) is called after mutations.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

REVIEW HOTSPOT **High-risk file**

Changes here should be reviewed with database state, transaction boundaries, and auth/session behaviour open side-by-side.

authenticate.js

Dimension	Description
Type	Middleware
Purpose	Verifies JWT signature and expiry, then compares JWT tv claim against current token_version.
Senior review checklist	Any future auth rewrite must preserve SESSION_REVOKED reason handling.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

REVIEW HOTSPOT **High-risk file**

Changes here should be reviewed with database state, transaction boundaries, and auth/session behaviour open side-by-side.

users.repo.js

Dimension	Description
Type	Auth repo patch
Purpose	Returns token_version and provides findTokenVersionByUserId for authenticate.
Senior review checklist	Keep query narrow: SELECT token_version FROM users WHERE user_id = ?.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

auth.service.js

Dimension	Description
Type	Auth service patch
Purpose	Embeds tv claim into issued JWT.
Senior review checklist	If refresh/login payload changes, keep tv included.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

adminUsers.validators.js

Dimension	Description
Type	Admin Users validator
Purpose	Validates list filters and mutation bodies. Deactivate reason is required.
Senior review checklist	Strict schemas prevent payload smuggling.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

adminUsers.stateMachine.js

Dimension	Description
Type	Admin Users state machine
Purpose	Pure guards for I-1 to I-5. Throws structured conflicts.
Senior review checklist	Never move invariants into controllers.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

adminUsers.repo.js

Dimension	Description
Type	Admin Users repo
Purpose	Owns SQL against users, user_roles, roles, cmms_emp_mst, cmms_section_mst, audit_log, user_role_history.
Senior review checklist	No caller above repo should know JOIN details.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

adminUsers.service.js

Dimension	Description
Type	Admin Users service
Purpose	Orchestrates role/status/force-logout transactions and cache invalidation.
Senior review checklist	Every mutation must bump token_version and write history/audit.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

REVIEW HOTSPOT **High-risk file**

Changes here should be reviewed with database state, transaction boundaries, and auth/session behaviour open side-by-side.

adminUsers.controller.js

Dimension	Description
Type	Admin Users controller
Purpose	Thin HTTP marshal layer.
Senior review checklist	No business rules here; route errors through next().
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

adminUsers.routes.js

Dimension	Description
Type	Admin Users routes
Purpose	Express paths with authenticate, authorize, validate, controller.
Senior review checklist	Order is the contract: identity, permission, schema, handler.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

employees.validators.js

Dimension	Description

Type	Employees validator
Purpose	Validates legacy employee form shapes and create-account role body.
Senior review checklist	Employee ID pattern must remain ^[A-Z]{2}[0-9]{5}$ unless project-wide rule changes.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

employees.repo.js

Dimension	Description
Type	Employees repo
Purpose	Aliases cmms_emp_mst columns to canonical fields and writes audit.
Senior review checklist	Do not leak EMM_* above this layer.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

employees.service.js

Dimension	Description
Type	Employees service
Purpose	Create/update/soft-delete employees and create user accounts with one-time password.
Senior review checklist	Create-account must return plaintext password only once and never store it.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

REVIEW HOTSPOT **High-risk file**

Changes here should be reviewed with database state, transaction boundaries, and auth/session behaviour open side-by-side.

employees.controller.js

Dimension	Description
Type	Employees controller
Purpose	Thin handlers for list/detail/create/update/delete/account.
Senior review checklist	Keep thin; do not add SQL or guard logic.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

employees.routes.js

Dimension	Description
Type	Employees routes
Purpose	Super Admin employee master route wiring.
Senior review checklist	All routes currently gated by master:employees:manage.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

server.js

Dimension	Description
Type	Server wiring
Purpose	Mounts Admin Users and Employees routers.
Senior review checklist	Mount order should preserve healthz and existing modules.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

adminUsers.js

Dimension	Description
Type	FE API wrapper
Purpose	Axios wrappers for admin user endpoints.
Senior review checklist	Return r.data.data consistently.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

employees.js

Dimension	Description
Type	FE API wrapper
Purpose	Axios wrappers for employee endpoints including create-account.
Senior review checklist	Do not store initial_password in localStorage or persistent state.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

adminUserSchemas.js

Dimension	Description
Type	FE schema
Purpose	Role constants and zod mirrors for role/status modals.
Senior review checklist	Keep role codes exactly in sync with roles table.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

employeeSchemas.js

Dimension	Description
Type	FE schema
Purpose	Create/update employee form zod schema.
Senior review checklist	Match backend widths and required fields.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

useAdminUserList.js

Dimension	Description
Type	FE hook
Purpose	SWR list cache for admin users.
Senior review checklist	Invalidate after role/status/force-logout actions.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

useEmployeeList.js

Dimension	Description
Type	FE hook
Purpose	SWR list cache for employees.
Senior review checklist	Invalidate after create/update/soft-delete/create-account.

Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

api-client.js

Dimension	Description
Type	FE infrastructure patch
Purpose	Handles SESSION_REVOKED as a security event.
Senior review checklist	Do not retry SESSION_REVOKED like a normal expired token.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

permissions.js

Dimension	Description
Type	FE navigation patch
Purpose	Adds SA-only Users and Employees admin nav entries.
Senior review checklist	Visibility must be permission-based, not role-string-based.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

UserList.jsx

Dimension	Description
Type	FE page
Purpose	User table, filters, role/status/force-logout modals.
Senior review checklist	Backend invariants still matter even if UI disables buttons.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

EmployeeList.jsx

Dimension	Description
Type	FE page
Purpose	Employee table, create-account modal, soft-delete button, edit links.
Senior review checklist	Warn clearly when initial_password is shown once.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

EmployeeForm.jsx

Dimension	Description
Type	FE page
Purpose	Shared create/edit form for employee master.
Senior review checklist	Employee ID immutable in edit mode.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

App.jsx

Dimension	Description
Type	FE routes
Purpose	Routes for /admin/users and /admin/employees including new/edit.

Senior review checklist	ProtectedRoute must wrap admin pages with required permissions.
Failure if wrong	Permission leakage, audit gap, stale JWT authority, employee-user inconsistency, or broken admin UX.

Appendix F — Endpoint Contract Reference

This appendix describes the intended endpoint behaviour without full source code. It can be used by frontend engineers, QA testers, and future API clients.

API Contract Summary

Endpoint	Input	Output / Side Effect
GET /admin/users	Query: q, role, is_active, division_id, sort, page, page_size.	Returns data.items and data.pagination. Each item includes id, employee_id, full_name, division, role, is_active, is_locked, token_version.
GET /admin/users/:id	Path id = user_id.	Returns one user detail including role, deactivation reason, token_version, and division snapshot.
GET /admin/users/:id/history	Path id = user_id.	Returns ordered transition rows from user_role_history.
PATCH /admin/users/:id/role	Body: role and optional reason.	Runs role guard, updates user_roles, bumps token_version, writes history and audit.
PATCH /admin/users/:id/activate	Body: optional reason.	Sets is_active=1, clears deactivation fields, bumps token_version, writes history and audit.
PATCH /admin/users/:id/deactivate	Body: reason min 5 chars.	Sets is_active=0, stores reason/by/at, bumps token_version, writes history and audit.
POST /admin/users/:id/force-logout	Body: optional reason.	Bumps token_version only, writes FORCE_LOGOUT history and audit.
GET /admin/employees	Query: q, is_active, division_id, has_account, sort, page, page_size.	Returns canonical employee rows with has_account and user_is_active.
POST /admin/employees	Body: employee master fields.	Inserts cmms_emp_mst row and writes audit.
PATCH /admin/employees/:id	Body: patch fields except employee_id.	Updates only supplied fields and writes audit.
DELETE /admin/employees/:id	No body.	Soft-deletes employee only if no active user account.
POST /admin/employees/:id/create-account	Body: optional role default NORMAL_USER.	Creates users and user_roles rows, writes CREATE history/audit, returns initial_password once.

API DESIGN LAW **Every mutation has a side effect beyond the immediate row**

Role/status/session mutations affect token_version and old sessions. Employee mutations affect audit_log. Account creation affects users, user_roles, history, audit, and returns one-time secret material.

Appendix G — Database Review Queries

These are light review-query ideas for senior validation. They are intentionally explanatory and not a replacement for automated tests.

```
REVIEW QUERIES

-- users now has token_version
SHOW COLUMNS FROM users LIKE "token_version";

-- verify Phase 7 permissions
SELECT permission_code FROM permissions
WHERE permission_code IN ("user:activate", "user:deactivate", "user:force-logout");

-- recent user transitions
SELECT action, from_role, to_role, from_active, to_active, reason, created_at
FROM user_role_history
ORDER BY created_at DESC
LIMIT 20;

-- detect users without user_roles row
SELECT u.user_id, u.employee_id
```

```
FROM users u LEFT JOIN user_roles ur ON ur.user_id = u.user_id
WHERE ur.user_id IS NULL;
```

Appendix H — Senior Code Review Checklist

Checklist

Area	Required check
Database	No migration contains DROP, RENAME, or destructive MODIFY.
Database	Every new column has a reason tied to Phase 7 behaviour.
Database	Every admin list query has appropriate indexes or accepted performance risk.
Auth	JWT payload includes tv and authenticate rejects stale tv.
Auth	SESSION_REVOKED is handled differently from normal expired token.
Admin Users	No role/status change can bypass adminUsers.service.
Admin Users	Every mutation writes user_role_history and audit_log inside one transaction.
Admin Users	Last active Super Admin cannot be demoted or deactivated.
Employees	cmms_emp_mst remains source of truth and is aliased in repo.
Employees	Employee soft-delete checks active user account before flipping EMM_INACTIVE.
Frontend	Modals use backend-driven outcome; UI disable is only convenience.
Frontend	initial_password is not persisted after modal close.
Routes	Every route follows authenticate -> authorize -> validate -> controller.
Verification	A1-A15 smoke transcript remains reproducible after changes.

Appendix I — Browser QA Script

Use this manual browser QA path when presenting Phase 7 Slice 1 to a mentor, stakeholder, or reviewer.

1. Login as SA79900.
2. Confirm Admin group appears in the sidebar with Users and Employees.
3. Open /admin/users and confirm table renders users, roles, status, and action icons.
4. Filter by role NORMAL_USER and confirm results change.
5. Open a role-change modal for a non-self user; select LAB_ENGINEER; save; confirm row updates.
6. Open that user history and confirm CHANGE_ROLE row exists.
7. Deactivate the same user with a reason; confirm status becomes inactive and history row appears.
8. Reactivate the user and confirm old session would be revoked due token_version bump.
9. Open /admin/employees and search an employee by ID or name.
10. Create a new test employee with a unique AC/TE-style ID.
11. Create account for that employee, choose role NORMAL_USER, copy initial password from one-time modal.
12. Attempt duplicate create-account for same employee and confirm HAS_ACCOUNT conflict.
13. Attempt soft-delete employee with active account and confirm it is blocked.
14. Deactivate corresponding user, then soft-delete employee if allowed by test setup.
15. Run npm run build and confirm zero FE errors.

Appendix J — Final Seal After Supplemental Deep Dive

SEALED **Phase 7 Slice 1 final statement**

This supplemental deep dive does not change implementation scope. It strengthens maintainability by documenting every file, endpoint, review rule, and browser QA path. Phase 7 Slice 1 remains sealed: DB + BE + FE + A1-A15 verification are complete.

END OF SUPPLEMENTAL DEEP DIVE · PHASE 7 SLICE 1 · SEALED